learned about the 15-Factor methodology, which contains two factors that help build observable applications. Factor 14 suggests treating your applications as space probes and reasoning about what kind of telemetry you'd need to monitor and control your applications remotely, such as logs, metrics, and traces. Factor 6 recommends treating logs as a stream of events rather than dealing with log files.

In this chapter, you'll learn how to ensure your Spring Boot applications expose relevant information to infer their internal states, such as logs, health probes, metrics, traces, and additional valuable data regarding schema migrations and builds. I'll also show you how to use the Grafana open source observability stack to validate the changes you'll make to your applications. However, I won't go into too many details, because that's something the platform team deploys and operates.

NOTE The source code for the examples in this chapter is available in the Chapter13/13-begin and Chapter13/13-end folders, which contain the initial and final states of the project (https://github.com/ThomasVitale/cloud-native-spring-in-action).

13.1 Logging with Spring Boot, Loki, and Fluent Bit

Logs (or *event logs*) are discrete records of something that happened over time in a software application. They are composed of a timestamp necessary to answer the question "when did the event happen?" and some information providing details about the event and its context, which lets us answer questions like "what happened at this time?", "which thread was processing the event?", or "which user/tenant was in the context?"

During troubleshooting and debugging tasks, logs are among the essential tools we can use to reconstruct what happened at a specific point in time in a